

Automated Assessment using AI-Enhanced Guided-Inquiry Learning Platform

Bo Kim¹, James Skon²

¹ Department of Computer Science
Southern New Hampshire University
Manchester, NH 03106
b.kim@snhu.edu

² Kenyon College
Gambier, OH 43022
skonjp@kenyon.edu

Abstract

This paper presents an assessment-mode extension to a guided-inquiry learning platform for introductory programming courses. The system delivers instructor-authored activities to learners, records their attempts (including code, explanations, and artifacts), and supports secure, containerized execution for C++ and Python with resource limits. During assessments, the platform enforces single-learner delivery and suppresses formative AI assistance. Submissions are routed through an AI first-pass grading step constrained by instructor directives and rubrics, producing concise, evidence-linked rationales. Instructors then review the evidence, adjust as needed, and publish final scores via an auditable interface. The design preserves pedagogical intent from guided inquiry, prioritizing reasoning and explanation, while improving turnaround and consistency relative to fully manual grading. We outline the platform’s architecture (clients, services, execution runners, data stores), describe the assessment flow from activity delivery through AI first-pass grading to instructor review and publication, and report early classroom experience highlighting strengths (rapid provisional feedback, evidence-linked rationales that support review, and alignment with course outcomes) as well as practical challenges (student expectations about provisional versus final scores, systematic edge cases in automated scoring and wording variability in short answers, and the need for question-specific numeric tolerances). We conclude by detailing planned extensions – targeted/batch AI regrading workflows, and more granular directive configuration at the assessment and question levels – alongside a formal evaluation of grading quality and student learning, including inter-rater reliability comparisons and timing analyses. The contribution is a practical blueprint for integrating AI-assisted grading into computing assessments that preserves instructor control and scales across varied class sizes.

Keywords: Introductory programming; AI-assisted assessment; Human-in-the-loop grading, AI-enabled learning platform

1. Introduction

Introductory programming courses, regardless of enrollment size, require assessments that are timely, consistent, and aligned with stated course outcomes. The core tension is speed versus pedagogical alignment. Traditional auto-grading accelerates turnaround but may misalign with pedagogical intent or over-emphasize surface features, while fully manual grading scales poorly and delays feedback.

We present an assessment-mode extension to the AI-enabled collaboration learning platform for C++ and Python that preserves instructor control while improving turnaround and consistency by (i) enforcing single-learner delivery, (ii) suppressing formative AI hints during the assessment window, (iii) executing an AI first-pass grading step constrained by instructor rubrics, visible/hidden tests, and assessment-level directive presets, and (iv) providing an audit-trailed instructor review before score release.

The platform delivers structured activities, records full attempts (code, explanations, artifacts), executes code in resource-constrained containers, and provides instructor-aligned AI first-pass grading that produces evidence-linked rationales for human review. This “human-in-the-loop” approach builds on research showing that active, collaborative, inquiry-oriented environments improve learning and retention [1]-[6] while acknowledging cautions about pedagogical alignment in AI-mediated feedback [7], [8], and the value of analytics for timely intervention [10].

Our contribution is threefold: (1) a system architecture and workflow for summative use that preserves instructor control; (2) configuration patterns for directive presets, rubric templates, and starter-code strategies; and (3) a deployment report from an introductory C++ course that motivates a forthcoming evaluation study.

2. Related Work and Pedagogical Foundations

Meta-analyses and foundational studies consistently show that active learning and structured collaboration improve student performance in STEM and engineering contexts [1], [4]. Within this broader landscape, guided-inquiry and other inductive approaches provide particularly coherent scaffolds for supporting concept formation and transfer by structuring how learners move from exploration to abstraction and application [2], [3], [5]. Building on these ideas, inquiry-based frameworks have also been adapted to introductory programming to structure multi-stage reasoning, sustain engagement, and help novices articulate and refine their mental models as they solve problems [6].

In parallel, work in AI in Education and intelligent tutoring systems highlights the potential of automated hints and formative feedback to support learning at scale, while also emphasizing a persistent challenge: feedback must remain aligned with instructor goals and delivered at appropriate times to avoid over-assistance, shortcutting productive struggle, or creating

mismatches with course pacing and assessment intent [7]-[9]. Against this backdrop, our stance is to operationalize guided inquiry in a digital setting by organizing activities around a progression from exploration to concept invention, application, and challenge. We additionally support a single-learner assessment mode by suppressing hints, preserving the reasoning-first design of guided inquiry while enabling faster, more consistent scoring with human oversight.

3. System Architecture

The architecture of our system translates the above-mentioned pedagogical priorities into a multi-service platform. **Figure 1** depicts the system architecture. A web client supports orchestration and instructor oversight, while backend services manage activity delivery and attempt workflows; submissions are executed in sandboxed, containerized C++/Python runners with resource limits, and a mediation component can generate concise, evidence-linked rationales under an assessment-level directive preset. In assessment mode, activities are structured as collections of atomic question units, each with its own rubric, points, runtime configuration, and (for code) visible/hidden tests. To preserve auditability, the assessment activity is snapshotted at delivery time and bound to each attempt.

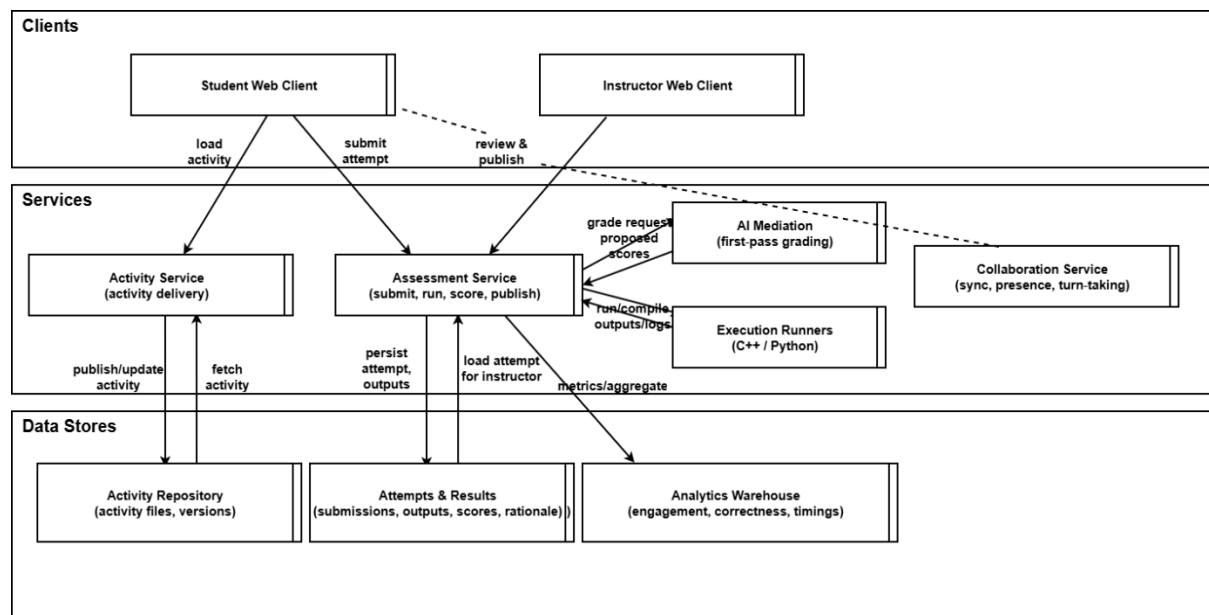


Figure 1. System Architecture and Data Flow

Figure 1 summarizes the resulting data flow. The student web client loads instructor-authored activities through the Activity Service, which serves versioned activity files from the Activity Repository. During work, students submit attempts to the Assessment Service, which dispatches code to sandboxed C++/Python execution runners and collects compile/run outputs and logs. When enabled, the Assessment Service also requests first-pass grading from the AI Mediation service, receiving proposed scores and (where applicable) evidence-linked rationales for subsequent instructor review. Attempt artifacts, proposed scores/rationales, and publish history

are persisted in Attempts & Results, while aggregated course-level signals (e.g., correctness, latency, and error patterns) are written to the Analytics Warehouse for monitoring. The instructor web client supports evidence-based review and publishing by loading attempts through the Assessment Service (retrieved from Attempts & Results) and updating or publishing activities through the Activity Service. In learning mode, a separate Collaboration Service provides synchronization, presence, and turn-taking; this layer can be disabled for individual assessment settings.

Activities (in learning mode) are structured documents with sections, question groups, prompts, optional rubrics, hidden directives, and executable code cells. In assessment mode, a single assessment-level directive preset governs AI behavior, and formative AI interactions are disabled during the attempt.

4. Assessment Mode Workflow

Assessment mode adapts the platform for individual exams by suppressing formative hints, locking collaboration tools, and channeling each submission through secure execution and instructor-aligned AI mediation before human review. Figure 2 outlines the sequence, which consists of the following steps.

- 1) Deliver activity (assessment mode) – Student loads an activity with hints suppressed.
- 2) Attempt authoring – Student edits code/explanations; optionally compiles/runs locally within resource limits.
- 3) Submit attempt – The client posts the attempt to the assessment service.
- 4) Secure execution – The service dispatches code to containerized runners; outputs/logs and exit codes return to the service.
- 5) AI first-pass grading – The service requests an AI evaluation constrained by instructor directives and rubrics; the AI returns proposed scores + concise, evidence-linked rationales.
- 6) Persist evidence – Attempt, outputs, and proposed grading evidence are stored.
- 7) Instructor review, then publish – Instructors review evidence, adjust as needed, and publish final scores; an audit trail records all actions.

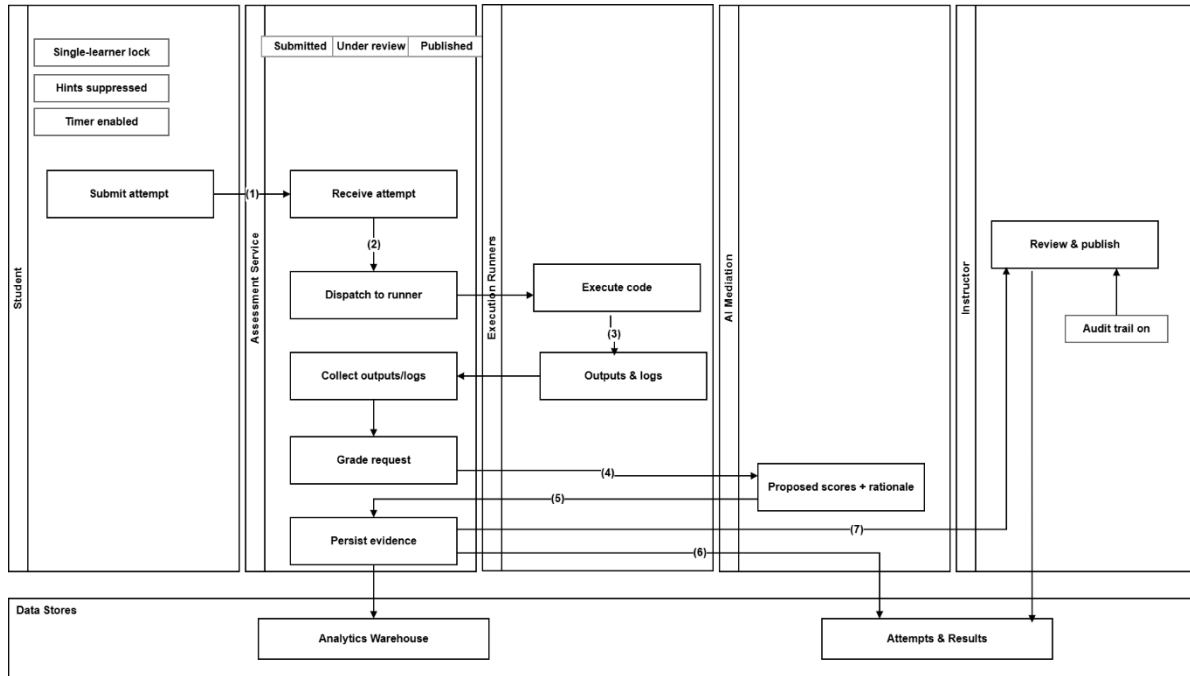


Figure 2. Assessment workflow

5. Implementation and Deployment

The web client is a modern single-page application integrated with the Activity and Assessment services. Execution runners are containerized (for C++ and Python) with strict CPU/memory/time limits. AI mediation is a service endpoint that applies assessment-level directive presets (e.g., *follow-ups disabled*, *requirements-only focus*, *concise rationales*). Data services persist attempts, outputs, rationales, and publish events for reproducibility and dispute resolution. In pilot use, instructors authored activities in a lightweight markup and Google Docs, then set assessment mode for quizzes/exams.

We provide directive presets, rubric templates, and starter-code strategies that reduce ambiguity and standardize summative behavior. Table 1 lists directive presets and their intended operational effects.

In summative use, directive presets are defined at the assessment level and apply uniformly across all questions. Variation in expectations is captured in each question’s rubric and tests.

Table 1. Assessment-level Directive Presets and Intended Effects

Preset	Operational Effect on Feedback	Intent
No follow-ups	Suppress follow-up questions during attempts	Maintain summative conditions
Requirements-only	Point to missing conditions without strategies	Preserve outcome alignment

Concise rationales	Single-sentence evidence-linked rationale	Reduce cognitive load; speed review
Checker errors do not block	Provide rationale but allow progression on minor issues	Maintain momentum where allowed
Match sample output exactly	Enforce exact labels/order on checkpoints	Support scoring reliability

Single-learner enforcement, time windows, attempt locking, and logs (compile/run rhythm, errors) support integrity while remaining compatible with in-room proctoring.

Whenever needed, starter code is used to minimize boilerplate. It saves time, reduces trivial syntax/setup errors, standardizes I/O for fair auto/AI-assist grading, and keeps attention on the learning target.

Hidden and visible tests for coding questions balance transparency and robustness.

6. Early Experience and Observations

We have conducted an initial experiment using the system in introductory programming class and two other CS classes, using the assessment workflow for several timed quizzes and one proctored exam composed of multiple questions.

From the initial experiment, we have observed the following benefits: faster turnaround vs fully manual grading; more consistent scoring; concise AI rationales standardized instructor review while preserving final authority.

We observed two practical workflow challenges. First, because the system can return immediate AI-generated rationales and provisional scores, students must be clearly informed that these outputs are preliminary and do not constitute the final grade; final scores are issued only after instructor review and publication. Without explicit messaging and UI cues, students may over-interpret the initial score as authoritative, which can undermine trust in the review process and generate unnecessary disputes.

Second, grading consistency becomes an operational concern when instructors observe a recurring misgrading pattern for a particular question (e.g., a rubric interpretation error or a systematic edge case). In such cases, instructors may wish to re-run AI grading in batch after adjusting the directive preset or rubric interpretation to produce updated proposed scores and rationales. The current system does not yet support this form of targeted or bulk AI regrading; instead, instructors must address these cases through manual review and correction.

7. Limitations and Future Work

For future work, we plan to address the observed assessment-mode challenges by improving robustness to legitimate variation in student responses and by reducing the instructor overhead

required to maintain grading consistency. On the grading side, we will extend the assessment pipeline to better accommodate valid but unexpected short-answer phrasings (e.g., via expanded paraphrase-aware matching and rubric-guided semantic checks) and to support question-specific numeric tolerances that can be tuned without rewriting the activity. We also plan to add targeted and batch AI regrading workflows so that when systematic issues are detected for a particular question, instructors can revise the relevant rubric, tolerance, or directive settings and re-run AI grading to regenerate proposed scores and evidence-linked rationales.

A second direction is to increase instructional control over AI behavior. At present, the system relies on a globally configured, hardcoded directive preset for all assessments; we aim to move directive configuration to the assessment level (and, where appropriate, the question/test level) to support finer-grained alignment with local grading intent. While this flexibility can improve alignment, it risks increasing authoring burden. To mitigate this, we will develop an instructor-facing authoring assistant that helps generate and validate activity specifications—including directives, rubrics, tests, and tolerance settings—through a guided interface. The goal is to let instructors focus on pedagogical content and assessment intent, while the tool handles the required tags, formatting, and consistency checks needed for reliable AI mediation and automated execution.

We have early classroom experience but not a causal evaluation. Future work includes a formal study of grading quality and learning outcomes, reporting inter-rater reliability and timing metrics (time from submit to publish, instructor review time), as well as exploring additional languages and different courses.

8. Conclusion

We describe a practical blueprint for integrating AI-assisted assessment into a guided-inquiry programming learning platform. By combining single-learner delivery, suppressed formative hints, secure execution, directive-bounded first-pass grading, and an auditable instructor review, the workflow scales to large sections without sacrificing alignment to pedagogical intent. The architecture and workflow of the system generalize across course sizes and support both C++ and Python. Deployment experience and a planned evaluation point to tangible gains in timeliness and consistency with clear next steps for rigor and control.

References

- [1] S. Freeman, S. L. Eddy, M. McDonough, M. K. Smith, N. Okoroafor, H. Jordt, and M. P. Wenderoth, “Active learning increases student performance in science, engineering, and mathematics,” *Proceedings of the National Academy of Sciences*, vol. 111, no. 23, pp. 8410–8415, 2014.
- [2] J. J. Farrell, R. S. Moog, and J. N. Spencer, “A guided-inquiry general chemistry course,” *Journal of Chemical Education*, vol. 76, no. 4, pp. 570–574, 1999.

- [3] R. S. Moog and J. N. Spencer, Eds., *Process Oriented Guided Inquiry Learning (POGIL)*, ACS Symposium Series, vol. 994. Washington, DC, USA: American Chemical Society, 2008.
- [4] D. W. Johnson, R. T. Johnson, and K. A. Smith, “Cooperative learning: Improving university instruction by basing practice on validated theory,” *Journal on Excellence in College Teaching*, vol. 25, nos. 3–4, pp. 85–118, 2014.
- [5] M. J. Prince and R. M. Felder, “Inductive teaching and learning methods: Definitions, comparisons, and research bases,” *Journal of Engineering Education*, vol. 95, no. 2, pp. 123–138, 2006.
- [6] S. Ahmed, S. A. H. Bukhari, and A. Ahmad, “Inquiry-based learning in software engineering education: Exploring students’ multiple inquiry levels in a programming course,” *Frontiers in Education*, vol. 10, 2025, Art. no. 1503996.
- [7] S. S. Lee and R. L. Moore, “Harnessing generative AI (GenAI) for automated feedback in higher education: A systematic review,” *Online Learning*, vol. 28, no. 3, pp. 82–104, 2024.
- [8] I. Roll and R. Wylie, “Evolution and revolution in artificial intelligence in education,” *International Journal of Artificial Intelligence in Education*, vol. 26, no. 2, pp. 582–599, 2016.
- [9] W. Holmes, M. Bialik, and C. Fadel, *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Boston, MA, USA: Center for Curriculum Redesign, 2019. ISBN: 978-1-794-29370-0.
- [10] O. H. T. Lu, A. Y. Q. Huang, J. C. H. Huang, A. J. Q. Lin, H. Ogata, and S. J. H. Yang, “Applying learning analytics for the early prediction of students’ academic performance in blended learning,” *Educational Technology & Society*, vol. 21, no. 2, pp. 220–232, 2018.